TOUCH CONTROL & RGB / PWM OUTPUT

Curiosity Kickoff

- What happens when you touch the sensor?
- Do you need to press hard?
- Does it work quickly?

Concept Connection

A Touch Control & RGB / PWM Output system uses a touch sensor to detect user input and control LED lighting. It employs PWM (Pulse Width Modulation) to adjust brightness and color of RGB LEDs, enabling smooth color changes, dimming, and interactive lighting control in electronic applications.

1.IR (Infrared) Sensor :

Detects infrared light, which is invisible to human eyes.Often used for obstacle detection or remote control.

Pinout: 3 pins - VCC: Power, GND: Ground & OUT: Digital signal

2. Watt SMD LED

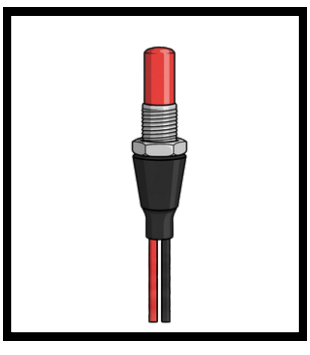
I'm flat and powerful. Just one of me can light up your whole table like a torch.I don't like wasting energy, so I shine super bright without getting too hot,You've already met me — I live in your mobile flashlight!

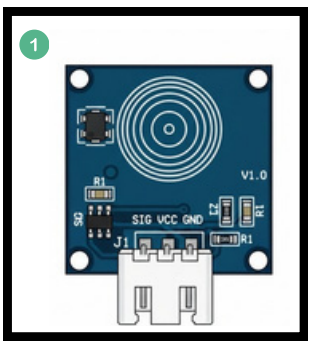
3.Multicolour 4-Leg LED – The Party Eye

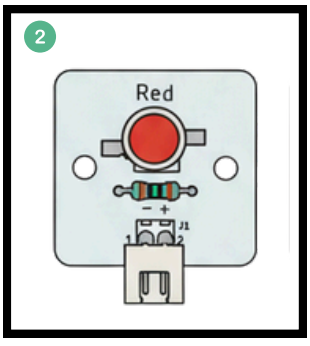
- I'm not just one color — I can glow red, green, or blue,Mix my colors together and I can create a rainbow inside your robot,I'm the one you'll call when you want to make your robot dance or throw a light show

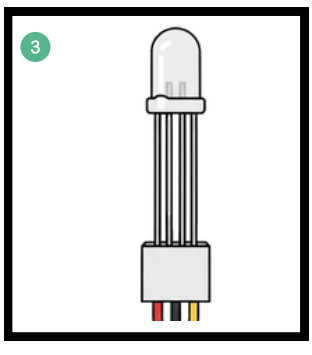
4.Push button

I may look small and simple, but with just one press I can make things happen. Push me once and I can wake your robot up, turn something on, or give a command. I'm the little switch your robot listens to when it needs a signal from you.









Beginner: Touch Sense

Class Explanation

A Touch Sense system uses a touch sensor to detect human touch input. When a user touches the sensor, it generates a signal that can control devices like LEDs or other outputs, enabling simple and interactive control in electronic applications.

Materials Required

- Arduino with attached shield
- 1 × Touch sensor
- 1 × white 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

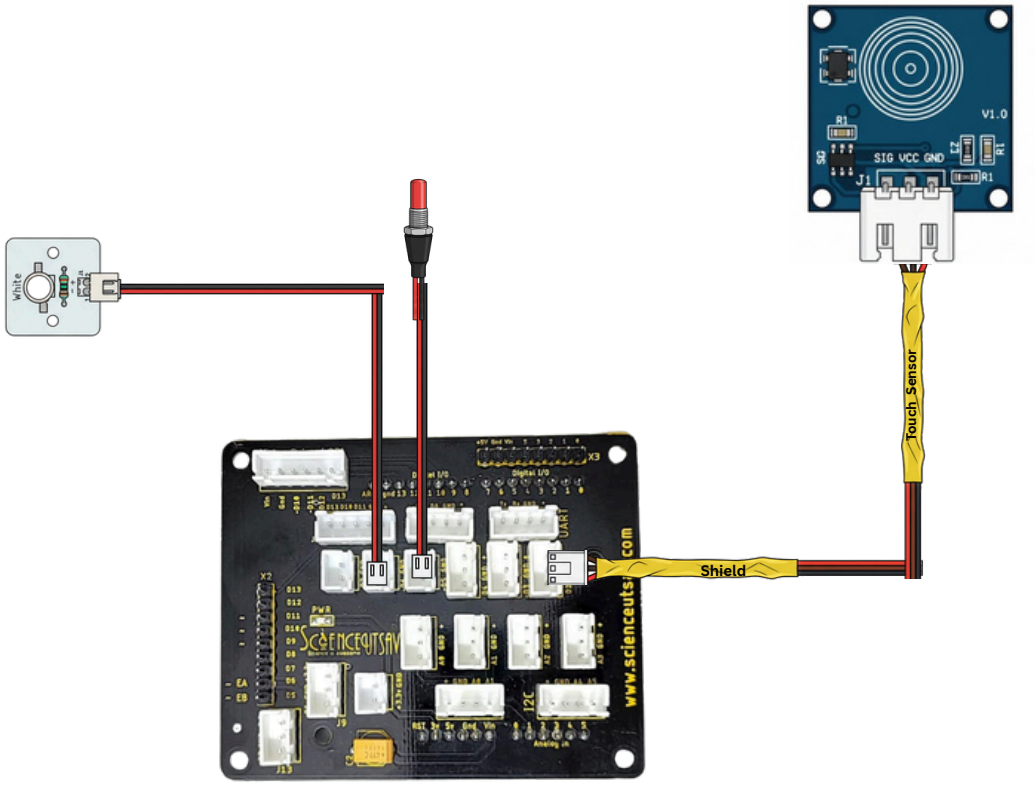
How to Connect

Use RMC wires to connect :

- 1 × touch sensor→ Arduino pin D2 and GND
- 1 × white 1 Watt SMD → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define TOUCH_PIN 2     // Touch sensor connected to D2
#define BUTTON_PIN 5    // Push button connected to D5
#define LED_PIN 6       // 1W LED connected to D6

bool ledState = false;  // Stores LED state

void setup() {
  pinMode(TOUCH_PIN, INPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // Button connected to GND when pressed
  pinMode(LED_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW); // LED initially OFF
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);
  int buttonState = digitalRead(BUTTON_PIN);

  // If touch sensor is touched, turn LED ON
  if (touchState == HIGH) {
    ledState = true;
  }

  // If push button is pressed, turn LED OFF
  if (buttonState == LOW) {   // because INPUT_PULLUP is used
    ledState = false;
  }

  // Update LED output
  digitalWrite(LED_PIN, ledState ? HIGH : LOW);
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 2 (TOUCH_PIN) and an LED (or buzzer) is connected to pin 3 (LED_PIN).
- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and the LED as OUTPUT using pinMode(). The LED is initially turned OFF using digitalWrite().
- The loop() function runs continuously. It reads the touch sensor value using digitalRead() and stores it in touchState.
- If the sensor is touched (HIGH signal), the LED turns ON. If not touched (LOW), the LED turns OFF.

So overall, the LED glows when the touch sensor is activated and turns OFF otherwise.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Touch Sense Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define TOUCH_PIN 2     // Touch sensor on D2

#define LED1_PIN 3      // First LED on D3
#define LED2_PIN 4      // Second LED on D4

void setup() {
  pinMode(TOUCH_PIN, INPUT);
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);

  if (touchState == HIGH) {
    digitalWrite(LED1_PIN, HIGH); // LED1 ON
    digitalWrite(LED2_PIN, HIGH); // LED2 ON
  } else {
    digitalWrite(LED1_PIN, LOW);  // LED1 OFF
    digitalWrite(LED2_PIN, LOW);  // LED2 OFF
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Touch Sense Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Touch Fade Controller

Class Explanation

In this class, students learn A Touch Fade Controller uses a touch sensor to control the brightness of an LED. When touched, it gradually increases or decreases the light intensity using PWM (Pulse Width Modulation), creating a smooth fading effect for interactive and energy-efficient lighting control.

Materials Required

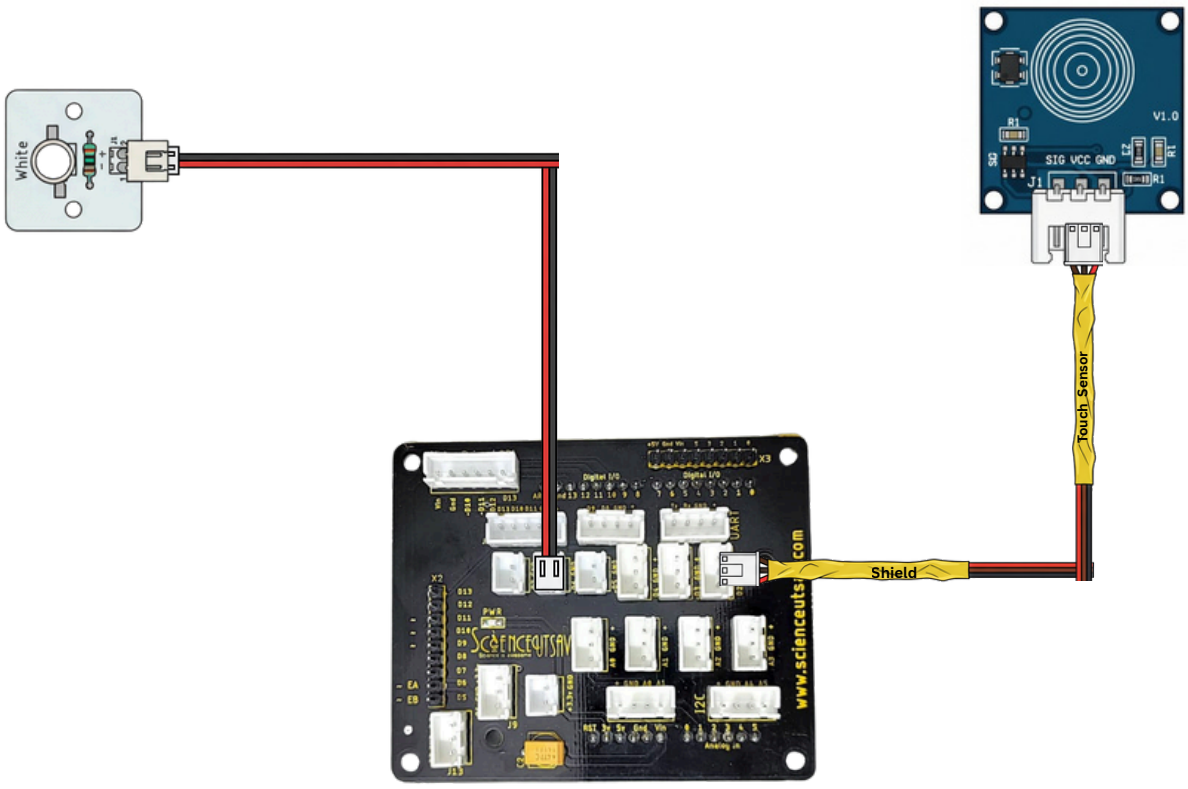
- Arduino with attached shield
- 1 × Touch sensor
- 1 × white 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

How to Connect

- Use RMC wires to connect :
- 1 × IR sensor→ Arduino pin D2 and GND
- 1 × white 1 Watt SMD LED → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define TOUCH_PIN 2 // Touch sensor connected to D2
#define LED_PIN 6  // LED connected to D6

void setup() {
 pinMode(TOUCH_PIN, INPUT);
 pinMode(LED_PIN, OUTPUT);
}

void loop() {
 int touchState = digitalRead(TOUCH_PIN);

 if (touchState == HIGH) {     // When touch sensor is pressed
  for (int brightness = 0; brightness <= 255; brightness++) {
   analogWrite(LED_PIN, brightness); // Gradually increase LED brightness
   delay(5);            // Smooth fade effect
  }
 } else {
  analogWrite(LED_PIN, 0);      // Turn LED OFF when not touched
 }
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 2 (TOUCH_PIN) and an LED is connected to pin 6 (LED_PIN).
- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and the LED as OUTPUT using pinMode().
- The loop() function runs continuously. It reads the touch sensor value using digitalRead() and stores it in touchState.
- If the sensor is touched (HIGH), a for loop is used to gradually increase the LED brightness from 0 to 255 using analogWrite(), creating a smooth fading effect.
- If the sensor is not touched (LOW), the LED is turned OFF by setting brightness to 0.

So overall, the LED slowly brightens when the touch sensor is activated and turns OFF when it is not touched.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Touch Fade Controller with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define TOUCH_PIN 2   // Touch sensor on D2

#define LED1_PIN 6    // LED1 on D6
#define LED2_PIN 5    // LED2 on D5
#define LED3_PIN 3    // LED3 on D3

void setup() {
  pinMode(TOUCH_PIN, INPUT);

  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(LED3_PIN, OUTPUT);
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);

  if (touchState == HIGH) {
    // Fade all LEDs together
    for (int brightness = 0; brightness <= 255; brightness++) {
      analogWrite(LED1_PIN, brightness);
      analogWrite(LED2_PIN, brightness);
      analogWrite(LED3_PIN, brightness);
      delay(5);
    }
  } else {
    // Turn all OFF
    analogWrite(LED1_PIN, 0);
    analogWrite(LED2_PIN, 0);
    analogWrite(LED3_PIN, 0);
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Touch Fade Controller with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced:Touch Activated LEDs

Class Explanation

In this class, students learn the A Touch Activated LEDs uses a touch sensor to detect user contact and control LED operation. When touched, the sensor triggers the LEDs to turn ON or OFF, providing a simple, user-friendly interface for controlling lighting in electronic projects and smart devices.

Materials Required

- Arduino with attached shield
- 1 × Touch sensor
- 2 × White 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

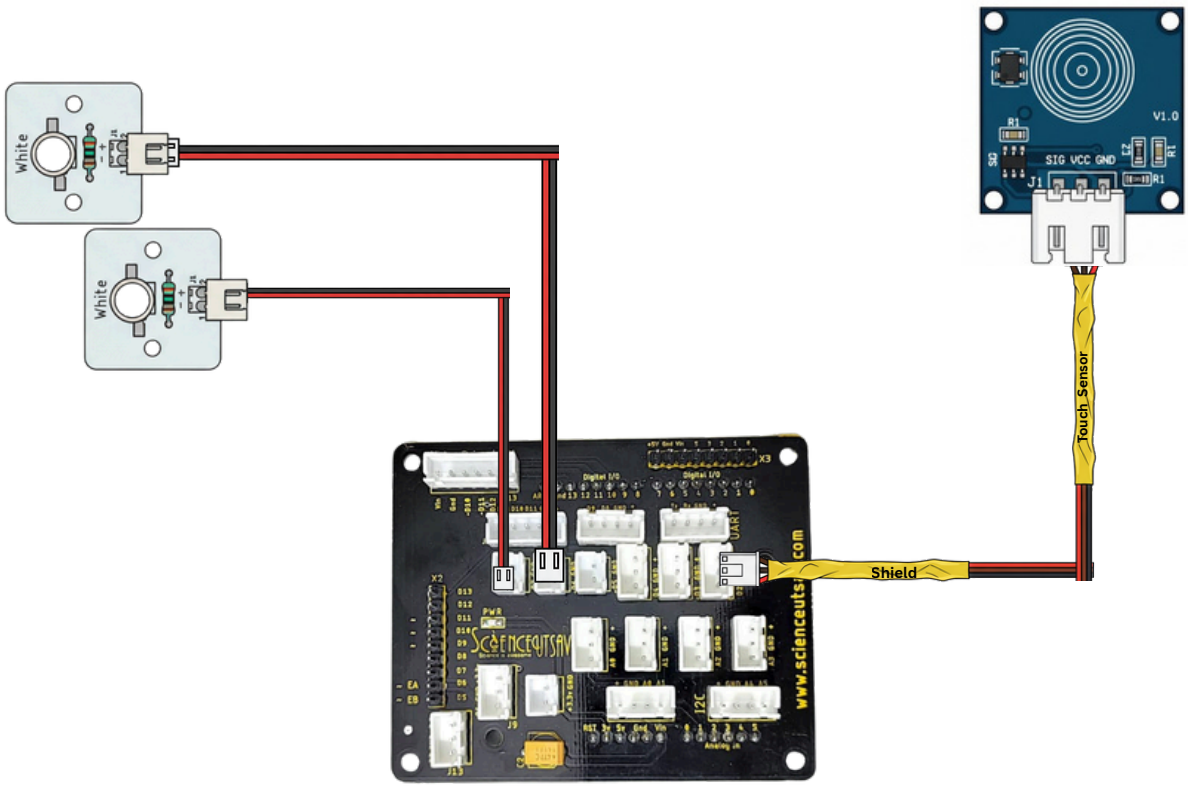
How to Connect

Use RMC wires to connect :

- 1 × IR sensor→ Arduino pin D2 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D6 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D7 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
// Touch Sensor with Two LEDs
// D6 LED lights when sensor output is LOW
// D7 LED lights when sensor output is HIGH

#define TOUCH_PIN 2 // Touch sensor connected to D2
#define LED_LOW 6  // LED for LOW state on D6
#define LED_HIGH 7  // LED for HIGH state on D7

void setup() {
 pinMode(TOUCH_PIN, INPUT); // Touch sensor input
 pinMode(LED_LOW, OUTPUT); // LED for LOW state
 pinMode(LED_HIGH, OUTPUT); // LED for HIGH state

 digitalWrite(LED_LOW, LOW);
 digitalWrite(LED_HIGH, LOW);
}

void loop() {
 int touchState = digitalRead(TOUCH_PIN);

 if (touchState == HIGH) {
  // Touch detected → turn ON LED_HIGH, OFF LED_LOW
  digitalWrite(LED_HIGH, HIGH);
  digitalWrite(LED_LOW, LOW);
 } else {
  // No touch → turn ON LED_LOW, OFF LED_HIGH
  digitalWrite(LED_HIGH, LOW);
  digitalWrite(LED_LOW, HIGH);
 }
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 2 (TOUCH_PIN), and two LEDs are connected to pins 6 (LED_LOW) and 7 (LED_HIGH). Each LED indicates a different state of the sensor.
- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and both LEDs as OUTPUT using pinMode(). Initially, both LEDs are turned OFF.
- The loop() function runs continuously. It reads the touch sensor value using digitalRead() and stores it in touchState.
- If the sensor is touched (HIGH), the LED on pin 7 turns ON and the LED on pin 6 turns OFF. If there is no touch (LOW), the LED on pin 6 turns ON and the LED on pin 7 turns OFF.

So overall, one LED indicates touch detection while the other shows when there is no touch.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Touch Activated LEDs but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define TOUCH_PIN 2   // Touch sensor connected to D2

#define LED_LOW 6     // LED for LOW state
#define LED_HIGH 7    // LED for HIGH state

#define BUZZER_PIN 5  // Buzzer on D5

void setup() {
  pinMode(TOUCH_PIN, INPUT);
  pinMode(LED_LOW, OUTPUT);
  pinMode(LED_HIGH, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LED_LOW, LOW);
  digitalWrite(LED_HIGH, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);

  if (touchState == HIGH) {
    // Touch detected
    digitalWrite(LED_HIGH, HIGH);
    digitalWrite(LED_LOW, LOW);
    digitalWrite(BUZZER_PIN, HIGH); // Buzzer ON
  } else {
    // No touch
    digitalWrite(LED_HIGH, LOW);
    digitalWrite(LED_LOW, HIGH);
    digitalWrite(BUZZER_PIN, LOW);  // Buzzer OFF
  }
}
```

Obective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Touch Activated LEDs how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Smart Touch Interface

Class Explanation

A Smart Touch Interface uses touch sensors to detect user input and control electronic devices. It allows smooth and responsive interaction, enabling functions like switching, dimming, or changing modes of LEDs and other outputs, making systems more intuitive, modern, and user-friendly.

Materials Required

- Arduino with attached shield
- 1 × Touch sensor
- 4 × White 1 Watt SMD LED on breakout boardRMC wires
- USB cable

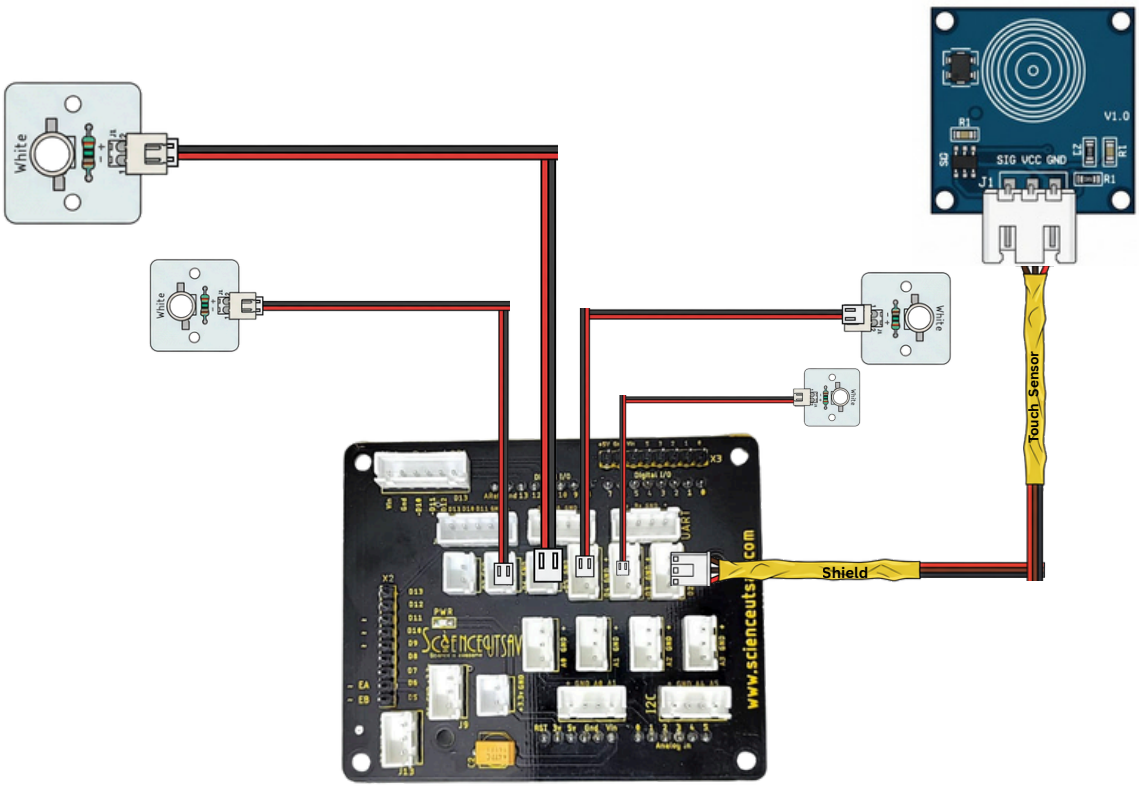
How to Connect

Use RMC wires to connect :

- 1 × Touch sensor → Arduino pin D2 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D3 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D4 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D5 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define TOUCH_PIN 2   // Touch sensor on D2

#define LED1 3        // LED1
#define LED2 4        // LED2
#define LED3 5        // LED3
#define LED4 6        // LED4

int count = 0;          // Store number of touches
int lastTouchState = LOW; // Store previous touch state

void setup() {
  pinMode(TOUCH_PIN, INPUT);

  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(LED3, OUTPUT);
  pinMode(LED4, OUTPUT);

  // All LEDs OFF initially
  digitalWrite(LED1, LOW);
  digitalWrite(LED2, LOW);
  digitalWrite(LED3, LOW);
  digitalWrite(LED4, LOW);
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);

  // Detect new touch (LOW → HIGH)
  if (touchState == HIGH && lastTouchState == LOW) {
    count++;   // Increase touch count

    if (count == 1) {
      digitalWrite(LED1, HIGH); // Turn ON LED1
    }
    else if (count == 2) {
      digitalWrite(LED2, HIGH); // Turn ON LED2
    }
```

```
 else if (count == 3) {
 digitalWrite(LED3, HIGH); // Turn ON LED3
 }
 else if (count == 4) {
 digitalWrite(LED4, HIGH); // Turn ON LED4
 }
 else {
 // Reset all LEDs
 digitalWrite(LED1, LOW);
 digitalWrite(LED2, LOW);
 digitalWrite(LED3, LOW);
 digitalWrite(LED4, LOW);

 count = 0; // Reset count
 }

 delay(200); // Debounce
 }

 lastTouchState = touchState; // Store current state
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 2 (TOUCH_PIN), and four LEDs are connected to pins 3, 4, 5, and 6 (LED1, LED2, LED3, LED4).

- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT and all LED pins as OUTPUT using pinMode(). Initially, all LEDs are turned OFF.

- The loop() function runs continuously. It reads the touch sensor value using digitalRead() and compares it with the previous state stored in lastTouchState.

- When a touch is detected (change from LOW to HIGH), the count value increases by 1. Based on the count, LEDs turn ON one by one:
- 1 → LED1 ON
- 2 → LED2 ON
- 3 → LED3 ON
- 4 → LED4 ON
- After the fourth touch, all LEDs turn OFF and the count resets to 0.

- A small delay() is added to avoid multiple detections from a single touch.

So overall, each touch lights up LEDs one by one, and resets after all LEDs are ON.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Smart Touch Interface but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define TOUCH_PIN 2     // Touch sensor on D2
#define BUZZER_PIN 5    // Buzzer on D5

int count = 0;             // Store touch count
int lastTouchState = LOW;  // Store previous state

void setup() {
  pinMode(TOUCH_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);
}

// Function to beep 'n' times
void beep(int times) {
  for (int i = 0; i < times; i++) {
    digitalWrite(BUZZER_PIN, HIGH); // ON
    delay(200);
    digitalWrite(BUZZER_PIN, LOW);  // OFF
    delay(200);
  }
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);

  // Detect touch (LOW → HIGH)
  if (touchState == HIGH && lastTouchState == LOW) {
    count++;
```

```
 if (count >= 1 && count <= 4) {
 beep(count); // Beep based on count
 } else {
 count = 0; // Reset after 4
 }

 delay(200); // Debounce
 }

 lastTouchState = touchState;
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn:
the basic idea of Smart Touch Interface how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- Understand how touch sensors detect human contact and generate signals to control electronic devices.
- Learn how touch input is used to gradually adjust LED brightness using PWM for smooth lighting control.
- Understand how touch sensors are used to turn LEDs ON and OFF for simple and interactive control systems.
- Learn how touch inputs are processed to control multiple functions, creating user-friendly and intelligent electronic interfaces.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output